

FORMATION EMBARQUÉE

Antisèches — sommaire

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Antisèches — mémos à imprimer

Des feuilles de référence **recto-verso**, à garder à côté du clavier. Elles ne remplacent pas les cours : elles évitent de rouvrir 200 pages pour retrouver une syntaxe ou une formule.

Antisèche	Pour	Quand la sortir
C embarqué	modules 01, 03, 10	dès que tu écris du C
C++ embarqué	module 02	classes, RAII, templates
VHDL	module 04, TP 2	tout le temps en FPGA
Arduino & STM32	modules 03, 10	API, broches, calculs de timer
Automates (Siemens/Schneider)	modules 07, 08	LADDER, SCL/ST, Modbus
Git & terminal	tout le parcours	les 15 commandes qui suffisent
Conversions & électronique	module 00	binaire/hexa, loi d'Ohm, niveaux

Conseil d'impression : en recto-verso, 2 pages par feuille, agrafées — tu obtiens un livret d'une dizaine de pages qui couvre toute la formation. Les PDF correspondants sont dans [../pdf/7-antiseches/](#).

FORMATION EMBARQUÉE

Antiséche — Arduino Stm32

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Antisèche — Arduino & STM32

Arduino — l'API en une page

```
void setup() { }           // une fois au démarrage
void loop() { }           // en boucle, indéfiniment

pinMode(p, OUTPUT | INPUT | INPUT_PULLUP);
digitalWrite(p, HIGH|LOW);  bool v = digitalRead(p);  // p appuyé = LOW en pull-up
int a = analogRead(A0);    // 0..1023 (Uno) · 0..4095 (ESP32)
analogWrite(p, 0..255);    // PWM (broches ~ seulement)

uint32_t t = millis();     // ms depuis le boot (déborde à 49,7 j)
uint32_t u = micros();     // µs (déborde à ~70 min)

Serial.begin(115200); Serial.print(x); Serial.println(F("en flash"));
if (Serial.available()) c = Serial.read();

attachInterrupt(digitalPinToInterrupt(2), isr, RISING|FALLING|CHANGE);
noInterrupts(); /* section critique */ interrupts();
map(v, 0,1023, 0,100); constrain(v, min, max);
```

Brochage Uno (l'essentiel)

Fonction	Broches
PWM	3, 5, 6, 9, 10, 11 (~)
Interruptions externes	2, 3
I2C	A4 = SDA, A5 = SCL
SPI	10 CS · 11 MOSI · 12 MISO · 13 SCK
UART (occupé par l'USB)	0 RX, 1 TX
Analogique	A0-A5 (10 bits)

⚠ 20 mA max par broche · LED = **résistance 220 Ω** · moteur = **driver** · alim séparée pour servos/moteurs (masses communes) · RAM = 2 Ko → **F()** partout, pas de **String**.

Le motif non bloquant (à recopier partout)

```
uint32_t dernier = 0;
void loop() {
  uint32_t now = millis();
  if (now - dernier >= PERIODE) { dernier = now; /* tâche */ }
  // autres tâches à d'autres périodes, aucune ne bloque
}
```

Anti-rebond

```
bool brut = (digitalRead(BTN) == LOW);
if (brut != brut_prec) { t_chgt = now; brut_prec = brut; }
if (now - t_chgt > 20 && brut != stable) { stable = brut; /* front */ }
```

STM32 — HAL en une page

```
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET|GPIO_PIN_RESET);
HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13);
uint32_t t = HAL_GetTick();          // ms - le millis() du STM32

HAL_UART_Transmit(&huart2, buf, len, 100);          // bloquant
HAL_UART_Receive_IT(&huart2, &octet, 1);           // IT - À RÉARMER !
HAL_UART_Transmit_DMA(&huart2, buf, len);           // DMA

HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, ccr);
__HAL_TIM_SET_AUTORELOAD(&htim2, arr);
HAL_TIM_Base_Start_IT(&htim3);

HAL_ADC_Start_DMA(&hadc1, (uint32_t*)tab, N);        // ADC circulaire
HAL_I2C_Mem_Read(&hi2c1, adr7 << 1, reg, 1, buf, n, 100); // << 1 !
HAL_SPI_TransmitReceive(&hspi1, tx, rx, n, 100);
```

Callbacks (à redéfinir, appelés depuis l'ISR)

```
void HAL_GPIO_EXTI_Callback(uint16_t pin);
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *h);
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *h); // + réarmer !
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *h);
```

Les formules de timer (à savoir refaire)

```
f_PWM = f_timer / ((PSC+1) × (ARR+1))
rapport cyclique = CCR / (ARR+1)
période = (PSC+1) × (ARR+1) / f_timer
```

Objectif (f_timer = 100 MHz)	PSC	ARR
1 kHz PWM	99	999
20 kHz PWM	4	999
IT toutes les 1 ms	99	999
Compteur en µs (1 tick = 1 µs)	99	65535

Les 3 modes d'E/S

Mode	Suffixe	CPU
Bloquant	(aucun)	occupé à attendre
Interruption	<code>_IT</code>	libre, ISR à la fin
DMA	<code>_DMA</code>	libre, transfert autonome

Accès registres (sans HAL)

```
RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN; // 1) TOUJOURS l'horloge d'abord
GPIOA->MODER &= ~GPIO_MODER_MODER5; // 2) nettoyer les bits
GPIOA->MODER |= GPIO_MODER_MODER5_0; // 3) écrire (01 = sortie)
GPIOA->ODR ^= GPIO_ODR_OD5; // 4) agir
```

Réflexes de débannage

Symptôme	Cause n°1
Périphérique muet	horloge RCC non activée
1 seul octet reçu en UART	<code>Receive_IT</code> non réarmé dans le callback
Valeur figée vue du main	<code>volatile</code> manquant
HardFault	pointeur nul/invalid → Fault Analyzer , regarder <code>BFAR</code> / <code>CFSR</code>
Plantage avec FreeRTOS	base de temps HAL sur SysTick, ou priorité NVIC trop haute
Comportement erratique	pile de tâche trop petite (<code>uxTaskGetStackHighWaterMark</code>)

FreeRTOS (CMSIS-v2) minimal

```
osThreadNew(TacheA, NULL, &attrA);
osDelay(100); // rend le CPU
osMessageQueuePut(q, &msg, 0, 0);
osMessageQueueGet(q, &msg, NULL, osWaitForever);
osMutexAcquire(m, osWaitForever); ... osMutexRelease(m);
// depuis une ISR : uniquement les variantes ...FromISR
```

FORMATION EMBARQUÉE

Antisèche — Automates

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Antisèche — Automates (Siemens & Schneider)

Le cycle automate

lire les entrées (image figée) → exécuter le programme → écrire les sorties en boucle, 1-10 ms. **Conséquence n°1** : ton code est rejoué des centaines de fois par seconde → sans **front**, un appui compte des centaines de fois.

Adressage

	Siemens (S7)	Schneider (Modicon)
Entrée bit	%I0.0	%I0.0
Sortie bit	%Q0.0	%Q0.0
Bit interne	%M0.0	%M0
Mot interne	%MW20	%MW20
Mot d'entrée ana.	%IW64	%IW0.0
Double mot / réel	%MD20 / Real	%MD20 / %MF20
Temporisation	FB TON + DB	%TM0
Compteur	FB CTU + DB	%C0
Système	—	%S6 (1 Hz), %SW

LADDER — les symboles

— — contact NO (vrai si bit = 1)	—()— bobine
— / — contact NF (vrai si bit = 0)	—(S)— set (mémoire 1)
— P — front montant (1 cycle)	—(R)— reset (mémoire 0)
— N — front descendant	

Marche/arrêt auto-maintenu (LE motif)

```
marche      arret      default      moteur
—| |———|/|———|/|———( )—
moteur |
—| |—      ← contact de maintien en parallèle
```

Verrouillage mutuel (2 sens, étoile/triangle...)

```
cmd_av      ar      ...      av      cmd_ar      av      ...      ar
—| |———|/|———( )—      —| |———|/|———( )—
```


Chaque sortie interdit l'autre — **et aussi en câblé** pour les risques de court-circuit (le logiciel seul ne suffit jamais).

Temporisations IEC

Bloc	Comportement
TON	Q passe à 1 après PT de IN vrai (retard à l'enclenchement)
TOF	Q retombe PT après que IN devienne faux
TP	impulsion calibrée de durée PT

CTU (comptage), CTD (décomptage), CTUD : PV = consigne, Q = atteint.

SCL / ST (texte structuré)

```
:= affectation      = <> < > <= >= comparaisons
AND OR NOT XOR      MOD ABS Sqrt

IF a > b THEN ... ELSIF ... ELSE ... END_IF;
FOR i := 0 TO 9 DO ... END_FOR;
WHILE cond DO ... END_WHILE;
CASE etape OF 0: ... 10: ... ELSE ... END_CASE;

#var                // variable locale du bloc (Siemens)
"DB".var            // variable globale/DB (Siemens)
T#2s500ms          // littéral de temps
INT_TO_REAL(x) REAL_TO_INT(x) DINT_TO_TIME(ms) // conversions explicites
```

Hystérésis (anti-battement) — à recopier

```
IF mesure < consigne - bande THEN sortie := TRUE;
ELSIF mesure > consigne + bande THEN sortie := FALSE;
END_IF; // entre les deux : on n'écrit RIEN, la sortie garde son état
```

GRAFCET en CASE (quand pas d'éditeur SFC)

```
CASE #etape OF
  0: IF depart THEN #etape := 10; END_IF;
  10: sortie_A := TRUE;
      IF capteur THEN #etape := 20; END_IF;
  20: #tempo(IN := TRUE, PT := T#5s);
      IF #tempo.Q THEN #tempo(IN := FALSE); #etape := 0; END_IF;
END_CASE;
```

Blocs Siemens

	Rôle
OB1	cycle principal · OB100 démarrage · OB3x cyclique
FC	fonction sans mémoire
FB + DB d'instance	fonction avec mémoire (≈ classe + objet)
DB	bloc de données global

Schneider : DFB (Control Expert) ≈ FB ; sections dans la tâche MAST .

Modbus

Zone	Lecture	Écriture
Coils (bits R/W)	01	05 (un), 15 (plusieurs)
Discrete inputs (bits R)	02	—
Holding registers (mots R/W)	03	06 (un), 16 (plusieurs)
Input registers (mots R)	04	—

- **RTU** : RS-485, 1 maître, esclaves 1-247, mêmes vitesse/parité partout, terminaisons aux extrémités.
- **TCP** : port **502**, adresse = numéro de mot (%MW100 → registre 100).
- Pièges : décalage « 40001 » des vieilles docs · ordre des mots pour les 32 bits (*word swap*) · **toujours borner** ce que le PC écrit.
- **Mot de vie** : compteur incrémenté par l'automate, surveillé par le PC — seule preuve que le programme tourne (une liaison TCP ouverte ne prouve rien).

Les règles de sûreté (non négociables)

1. **Arrêt d'urgence en NF** (sécurité positive : fil coupé = déclenché).
2. **Réarmement volontaire** : disparition de la cause **ET** acquit opérateur.
3. **Une seule section écrit les sorties**, et la sécurité y est **en aval** : `KM1 := (auto OR manu) AND aucun_defaut;`
4. **Surveillance de mouvement** : pas de fin de course atteinte en N s → défaut.
5. **La sécurité ne vit jamais dans l'IHM** ni dans une séquence qui peut se bloquer.
6. Bouton HMI à **impulsion** ; le maintien appartient à l'automate.
7. L'état de repli se déduit du **risque du procédé** (couper le chauffage, mais ventiler à fond).

FORMATION EMBARQUÉE

Antisèche — C Embarque

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Antisèche — C embarqué

Types (toujours explicites)

```
#include <stdint.h>
uint8_t u8;      // 0..255           int8_t s8;      // -128..127
uint16_t u16;    // 0..65535         int16_t s16;    // -32768..32767
uint32_t u32;    // 0..4 294 967 295 int32_t s32;
bool b;         // <stdbool.h>      size_t taille;
```

⚠ `int` = 16 bits sur AVR/Arduino, 32 bits sur ARM → jamais de `int` nu pour une valeur qui compte.
Pas de `float` sans FPU : **point fixe** (stocker des centièmes en `int32_t`).

Les 5 opérations de bits (à savoir de tête)

```
r |= (1u << n);      // SET      mettre le bit n à 1
r &= ~(1u << n);     // CLEAR   mettre le bit n à 0
r ^= (1u << n);      // TOGGLE  inverser le bit n
if (r & (1u << n)) {} // TEST    le bit n est-il à 1 ?
champ = (r >> pos) & msk; // EXTRAIRE un champ (décaler PUIS masquer)

r = (r & ~(0x3u << 2)) | (val << 2); // écrire un champ sans toucher au reste
```

Masques utiles

<code>0x0F</code> bits 0-3	<code>0xF0</code> bits 4-7
<code>0xFF</code> un octet	<code>0xFFFF</code> deux octets
<code>x & 0xFF</code> garder l'octet bas	<code>(x >> 8) & 0xFF</code> prendre l'octet haut
<code>(hi << 8) lo</code> assembler 16 bits	<code>1u << n</code> bit n seul

Pointeurs

```
uint8_t x = 42;
uint8_t *p = &x; // p contient l'ADRESSE de x
*p = 100;        // écrit DANS x
p[2] == *(p + 2); // arithmétique : avance par taille d'élément

const uint8_t *p; // données non modifiables via p
uint8_t *const p; // adresse figée
volatile uint32_t *reg; // registre matériel (le cas standard)

#define GPIOC_ODR (*(volatile uint32_t *)0x4001100C) // accès registre
```

Pièges mortels : pointeur non initialisé · retour d'adresse d'une variable locale · `strcpy` sans borne (utiliser `snprintf`).

| volatile — et ses limites

Obligatoire pour : registres matériels · variables partagées avec une ISR · variables partagées entre tâches RTOS.

```
volatile uint8_t drapeau; // écrit par l'ISR, lu par main
```

⚠ **volatile** ≠ **atomique**. Un `uint32_t` sur CPU 8 bits se lit en 4 fois → section critique :

```
noInterrupts(); copie = compteur; interrupts(); // Arduino
__disable_irq(); copie = compteur; __enable_irq(); // ARM
```

| Machine d'états (le patron n°1)

```
typedef enum { REPOS, ACTIF, ERREUR } Etat;
static Etat etat = REPOS;
static uint32_t t_entree;

void tick(uint32_t maintenant) {
    switch (etat) {
        case REPOS:
            if (demarrer) { etat = ACTIF; t_entree = maintenant; }
            break;
        case ACTIF:
            if (maintenant - t_entree >= DUREE) { etat = REPOS; ... }
            break; // ↑ soustraction : robuste au wrap
        case ERREUR: securiser(); break;
    }
}
```

| Temps non bloquant

```
if (maintenant - dernier >= PERIODE) { dernier = maintenant; /* action */ }
```

✓ `maintenant - dernier >= P` ✗ `maintenant >= dernier + P` (casse au débordement à 49,7 jours pour un `uint32_t` en ms).

| Structures & trames

```
typedef struct { uint8_t id; int16_t val; } Mesure;
Mesure m = { .id = 1, .val = 253 };
Mesure *pm = &m; pm->id = 2;
```

⚠ **Padding** : `sizeof(struct)` ≥ somme des champs. Ne JAMAIS caster un buffer réseau en struct → décoder champ par champ :

```
val = (int16_t)(((uint16_t)buf[2] << 8) | buf[3]); // big-endian
```

Squelette bare-metal & règles d'or

```
int main(void) {  
    init_materiel();  
    while (1) { /* lire → décider → agir, jamais bloquant */ }  
}
```

1. Compiler `-Wall -Wextra -Werror`, zéro warning.
2. Pas de `malloc` après l'init (fragmentation, échec imprévisible).
3. ISR : courte, un drapeau, pas de `printf` / `delay` / allocation.
4. Vérifier chaque retour de fonction d'E/S.
5. Tester les bords : 0, max, débordement, négatif, entrée invalide.

Compilation

```
gcc -Wall -Wextra -O2 main.c -o app # PC  
avr-gcc -mmcu=atmega328p -DF_CPU=16000000UL -Os blink.c -o blink.elf  
arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -Os main.c  
gcc -S main.c # voir l'assembleur godbolt.org en ligne
```

FORMATION EMBARQUÉE

Antisèche — Conversions

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Antisèche — Conversions & électronique

Table binaire / hexa / décimal

Déc	Bin	Hex	Déc	Bin	Hex
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	10	1010	A
3	0011	3	11	1011	B
4	0100	4	12	1100	C
5	0101	5	13	1101	D
6	0110	6	14	1110	E
7	0111	7	15	1111	F

Méthode : grouper le binaire **par 4 en partant de la droite**, chaque groupe = 1 chiffre hexa. **1011 0100** → **B4** → **0xB4** = $11 \times 16 + 4 = 180$.

Puissances de 2

n	2 ⁿ	n	2 ⁿ
0	1	8	256
1	2	10	1 024 (1 Ki)
2	4	12	4 096
3	8	16	65 536
4	16	20	1 048 576 (1 Mi)
5	32	24	16 777 216
6	64	31	2 147 483 648
7	128	32	4 294 967 296

Plages des types

Bits	Non signé	Signé (complément à 2)
8	0 ... 255	-128 ... +127
12 (ADC)	0 ... 4 095	—
16	0 ... 65 535	-32 768 ... +32 767
32	0 ... 4 294 967 295	-2 147 483 648 ... +2 147 483 647

Complément à deux : inverser tous les bits, ajouter 1. $+10 = 0000\ 1010 \rightarrow 1111\ 0101 \rightarrow 1111\ 0110 = -10 = 0xF6$. Vérif : lu en non signé, $0xF6 = 246 = 256 - 10$. MSB à 1 $\Rightarrow$ négatif.

Conversions ADC / DAC

```
tension = brut × Vref / (2n - 1)
brut = tension × (2n - 1) / Vref
résolution (1 LSB) = Vref / 2n
```

Cible	n	Vref	1 LSB
Arduino Uno	10 bits (0-1023)	5 V	≈ 4,9 mV
ESP32 / STM32	12 bits (0-4095)	3,3 V	≈ 0,8 mV

Point fixe (sans flottant) : $mV = (brut * 3300UL) / 4095;$

Industriel

```
4-20 mA : valeur = (I_mA - 4) / 16 × étendue
0-10 V : valeur = U / 10 × étendue
Siemens analogique : 0..27648 pour la pleine échelle
Convention du cours : entiers ×10 (253 = 25,3 °C) – jamais de Real en Modbus
```

Temps, fréquence, débit

```
T = 1/f          f = 1/T
1 kHz = 1 ms · 1 MHz = 1 µs · 100 MHz = 10 ns par cycle
```

UART 8N1	10 bits transmis par octet
9 600 bauds	1 bit ≈ 104 µs · 960 octets/s
115 200 bauds	1 bit ≈ 8,7 µs · 11 520 octets/s

débit utile (o/s) = baudrate / 10

Débordement des compteurs ms 32 bits : 2^{32} ms $\approx$ **49,7 jours** → toujours maintenant - dernier $\geq$ période .

Loi d'Ohm & résistance de LED

$$U = R \times I \quad P = U \times I$$
$$R_{LED} = (V_{alim} - V_{LED}) / I_{LED}$$

Exemple : 5 V, LED rouge (≈ 2 V), 10 mA → $R = (5-2)/0,01 = \mathbf{300\ \Omega}$ → on prend 330 Ω (valeur normalisée supérieure). **220 Ω** est le classique (≈ 13 mA, plus lumineux, toujours dans les 20 mA/broche).

Tension LED typique rouge 1,8-2,2 V · vert/jaune 2,0-2,4 V · bleu/blanc 3,0-3,4 V

Niveaux logiques et bonnes pratiques

Famille	0 logique	1 logique
TTL 5 V (Uno)	< 0,8 V	> 2,0 V
CMOS 3,3 V (STM32, ESP32, Pi)	< 0,8 V	> 2,0 V

⚠ **Ne jamais injecter 5 V sur une broche 3,3 V** (sauf « 5 V tolerant » explicite) → pont diviseur ou convertisseur de niveau. **Pull-up** typique : 10 k Ω (bouton), **4,7 k Ω** (I2C). Une entrée non câblée **flotte** : toujours un pull-up/pull-down. Découplage : **100 nF** au plus près de chaque alimentation de circuit.

Code couleur des résistances (4 anneaux)

Couleur	Chiffre	Multiplieur
noir	0	$\times 1$
marron	1	$\times 10$
rouge	2	$\times 100$
orange	3	$\times 1\text{ k}$
jaune	4	$\times 10\text{ k}$
vert	5	$\times 100\text{ k}$
bleu	6	$\times 1\text{ M}$
violet	7	—
gris	8	—
blanc	9	—
or	—	tolérance $\pm 5\%$

220 Ω = rouge-rouge-marron · 1 kΩ = marron-noir-rouge · 4,7 kΩ = jaune-violet-rouge · 10 kΩ = marron-noir-orange.

Préfixes

p	n	μ	m	—	k	M	G
10 ⁻¹²	10 ⁻⁹	10 ⁻⁶	10 ⁻³	1	10 ³	10 ⁶	10 ⁹

FORMATION EMBARQUÉE

Antisèche — C++ Embarque

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Antisèche — C++ embarqué

Le « C++ embarqué raisonnable » : classes, RAII, templates, `constexpr`, références. **Sans** exceptions, RTTI, ni allocation dynamique imprévisible (`-fno-exceptions -fno-rtti`).

Ce que C++ ajoute tout de suite

```
bool ok = true;           // vrai booléen
int& ref = n;             // référence : alias toujours valide
void f(const Measure& m);  // passage sans copie
auto x = 42u;             // déduction de type
constexpr uint32_t F = 16'000'000; // calculé à la COMPILE
static_assert(F % 1000 == 0, "message"); // vérifié à la compilation
enum class Mode : uint8_t { Repos, Actif }; // typée, pas de fuite de noms
nullptr                  // au lieu de NULL
```

Classe type

```
class Led {
public:
    explicit Led(uint8_t broche) : broche_(broche) { // liste d'init
        pinMode(broche_, OUTPUT);
    }
    ~Led() { eteindre(); } // destructeur

    void allumer() { digitalWrite(broche_, HIGH); on_ = true; }
    bool estAllumee() const { return on_; } // const = ne modifie rien

    Led(const Led&) = delete; // interdire la copie
    Led& operator=(const Led&) = delete;

private:
    uint8_t broche_;
    bool on_ = false; // init par défaut
};
```

RAII — l'idée maîtresse

Ressource acquise dans le **constructeur**, libérée dans le **destructeur** : libération **garantie** sur tous les chemins de sortie (return anticipé compris).

```

class SectionCritique {
public:
    SectionCritique() { noInterrupts(); }
    ~SectionCritique() { interrupts(); }
    SectionCritique(const SectionCritique&) = delete;
};

void f() {
    SectionCritique sc;          // ← IRQ coupées
    if (erreur) return;          // ← elles seront QUAND MÊME rétablies
}                                // ← ici aussi

```

Usages : section critique · chip-select SPI · mutex RTOS · transaction I2C.

Héritage & polymorphisme (drivers interchangeable)

```

class ICapteur {
public:
    virtual ~ICapteur() = default;          // OBLIGATOIRE si héritage
    virtual int16_t lire() = 0;             // = 0 → classe abstraite
};

class Bme280 : public ICapteur {
public:
    int16_t lire() override { return ...; } // override = vérifié
};

ICapteur* tab[] = { &bme, &ds18b20 };
for (ICapteur* c : tab) log(c->lire());    // la bonne méthode est appelée

```

Coût : une vtable (1 pointeur/objet + 1 indirection). L'alternative sans coût = **templates** (résolu à la compilation).

Templates

```

template <typename T, size_t N>
class Tampon {
    static_assert((N & (N-1)) == 0, "N doit être une puissance de 2");
public:
    bool put(const T& v);
    bool get(T& v);
private:
    T buf_[N] {};                // stockage STATIQUE, zéro malloc
    volatile size_t tete_ = 0, queue_ = 0;
};

Tampon<uint8_t, 64> rx;          // instancié à la compilation

template <typename T>
constexpr T borner(T v, T lo, T hi) { return v < lo ? lo : (v > hi ? hi : v); }

```

STL en embarqué

✓ Utilisable sur micro

`std::array<T,N>`

`std::optional`, `std::pair`

`<algorithm>` (`min`, `max`, `clamp`, `sort`)

`std::atomic` (si supporté)

✗ À éviter (allocation dynamique)

`std::vector`, `std::string`, `std::map`

`std::function` (peut allouer)

`iostream` (très lourd)

exceptions, RTTI

Sur Linux embarqué (Pi, i.MX) : toute la STL est utilisable.

Lambdas (callbacks)

```
bouton.surAppui([]() { led.basculer(); }); // sans capture
auto seuil = 100;
filtrer([seuil](int16_t v) { return v > seuil; }); // capture par valeur
```

Interopérer avec du C (HAL, drivers constructeur)

```
extern "C" {
#include "stm32f4xx_hal.h" // empêche le name mangling
}
extern "C" void HAL_UART_RxCpltCallback(UART_HandleTypeDef *h) { }
```

Pièges spécifiques

1. Les **objets globaux** ont leur constructeur exécuté **avant** `main()` → ne pas y toucher au matériel non initialisé.
2. `new` / `delete` : mêmes réserves que `malloc` (fragmentation).
3. Un destructeur **non virtuel** dans une classe de base = fuite/UB.
4. Copie implicite d'un objet qui détient une ressource → `= delete`.
5. `std::string` par valeur en paramètre = copie + allocation → `const&`.

FORMATION EMBARQUÉE

Antisèche — Git Terminal

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Antisèche — Git & terminal

Git : les 12 commandes qui suffisent

```
git init                # créer un dépôt dans le dossier courant
git clone <url>          # récupérer un dépôt existant
git status              # QUE se passe-t-il ? (à taper tout le temps)
git add fichier.c / git add .      # préparer (stager)
git commit -m "message clair"      # enregistrer
git log --oneline -10             # historique court
git diff / git diff --staged      # ce qui a changé
git push / git pull              # envoyer / récupérer
git branch ma-branche && git switch ma-branche # (ou checkout -b)
git restore fichier.c            # annuler mes modifs NON commitées
git revert <sha>                 # annuler un commit déjà partagé (propre)
```

Messages de commit utiles

```
TP1 seance 2 : OLED + decoupage en modules
fix: reamorcner Receive_IT (on ne recevait qu'un octet)
TD 04 ex.3 : anti-rebond + synchroniseur, testbench vert
```

Impératif présent, court, ce qui change **et pourquoi**. Un commit par séance de travail minimum.

Se sortir des ennuis

```
git commit --amend -m "message corrigé"    # corriger le DERNIER commit (non poussé)
git reset --soft HEAD~1                    # défaire le dernier commit, garder les fichiers
git stash / git stash pop                   # mettre de côté / reprendre
git checkout -- .                           # tout jeter (Δ irréversible)
```

`.gitignore` type pour l'embarqué

```
*.o
*.elf
*.hex
*.bin
*.map
Debug/
build/
__pycache__/
*.class
*.cf          # GHDL
*.ghw         # ondes GTKWave
.vscode/
```

Terminal Linux/macOS — le minimum vital

```
pwd                # où suis-je ?
ls -la             # lister (avec cachés + détails)
cd dossier / cd .. / cd ~
mkdir -p a/b/c     # créer une arborescence
cp src dst / mv src dst / rm fichier / rm -r dossier
cat f.txt / less f.txt / head -20 f / tail -f journal.log
grep -rn "motif" .  # chercher dans les fichiers
find . -name "*.c"  # chercher des fichiers
chmod +x script.sh  # rendre exécutable
```

Série & matériel

```
ls /dev/tty*       # trouver le port (ttyUSB0, ttyACM0)
dmesg | tail       # "quel port ma carte vient-elle de prendre ?"
sudo usermod -aG dialout $USER # droits port série (Linux, puis se reconnecter)

picocom -b 115200 /dev/ttyACM0 # terminal série (quitter : Ctrl-A Ctrl-X)
screen /dev/ttyACM0 115200     # (quitter : Ctrl-A puis K)
minicom -D /dev/ttyUSB0 -b 115200
```

Sous **Windows** : PuTTY ou le moniteur série de l'IDE (port **COMx**).

Compilation & outils

```
make                # construire      make clean / make test
gcc -Wall -Wextra -O2 f.c -o app
hexdump -C firmware.bin | head    # inspecter un binaire
arm-none-eabi-objdump -d f.elf     # désassembler
arm-none-eabi-size f.elf           # taille flash/RAM utilisée
```

Python (bancs de test, Modbus, tracés)

```
python3 -m venv .venv && source .venv/bin/activate # env isolé
pip install pyserial pymodbus matplotlib
python3 script.py
```

```
import serial                # parler à la carte
s = serial.Serial('/dev/ttyACM0', 115200, timeout=1)
s.write(b'status\n'); print(s.readline().decode())
```

Raccourcis terminal qui font gagner du temps

Touches	Effet
<code>Tab</code>	complète le nom (le réflexe n°1)
<code>↑</code> / <code>Ctrl-R</code>	commande précédente / recherche dans l'historique
<code>Ctrl-C</code>	interrompre le programme en cours
<code>Ctrl-A</code> / <code>Ctrl-E</code>	début / fin de ligne
<code>Ctrl-L</code>	effacer l'écran
<code>!!</code>	rejouer la dernière commande (<code>sudo !!</code>)

FORMATION EMBARQUÉE

Antisèche — Vhdl

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Antisèche — VHDL

Rappel permanent : le VHDL **décrit un circuit**, il ne s'exécute pas. Question à se poser à chaque ligne : *quel matériel je fabrique là ?*

Squelette

```
library ieee;
use ieee.std_logic_1164.all;    -- std_logic, std_logic_vector
use ieee.numeric_std.all;      -- unsigned, signed, arithmétique

entity mon_bloc is
  generic ( LARGEUR : natural := 8 );
  port (
    clk   : in  std_logic;
    reset : in  std_logic;
    d     : in  std_logic_vector(LARGEUR-1 downto 0);
    q     : out std_logic_vector(LARGEUR-1 downto 0)
  );
end entity;

architecture rtl of mon_bloc is
  signal interne : unsigned(LARGEUR-1 downto 0) := (others => '0');
begin
  -- ici les process et affectations concurrentes
end architecture;
```

Types et conversions

```
signal b : std_logic;           -- '0' '1' 'Z' 'U' 'X' '-'
signal v : std_logic_vector(7 downto 0); -- bus "neutre" (interfaces)
signal u : unsigned(7 downto 0); -- sait compter
signal s : signed(11 downto 0);  -- complément à 2
signal n : integer range 0 to 99; -- compteur borné

v <= "10100101";    v <= x"A5";      -- littéraux
u <= u + 1;          -- OK (pas sur slv !)
v <= std_logic_vector(u);           -- conversions EXPLICITES
u <= unsigned(v);
n <= to_integer(u);  u <= to_unsigned(n, 8);
v <= (others => '0'); -- tout à zéro (quelle que soit la largeur)
```

Les 2 process canoniques

```
-- SÉQUENTIEL (registres) : la forme la plus fréquente
process (clk)
begin
    if rising_edge(clk) then
        if reset = '1' then q <= (others => '0');
        elsif en = '1' then q <= d;
        end if;
    end if;
end process;

-- COMBINATOIRE : liste de sensibilité COMPLÈTE + valeur par défaut
process (a, b, sel)
begin
    y <= '0'; -- défaut : évite le latch !
    if sel = '1' then y <= a; else y <= b; end if;
end process;
```

Concurrent (hors process)

```
y <= a and b; -- porte
y <= a when en = '1' else '0'; -- mux conditionnel
with sel select
    y <= a when "00", b when "01", c when others; -- others OBLIGATOIRE
```

Les 5 règles qui évitent 90 % des bugs

1. Un signal n'est piloté que **dans un seul** process (sinon *multiple drivers*).
2. Process combinatoire : sortie affectée dans **tous** les chemins → sinon **latch**.
3. Toute entrée **asynchrone** passe par **2 bascules** (métastabilité).
4. **signal** (<=) se met à jour en fin de delta ; **variable** (:=) immédiatement.
5. Une seule horloge, front montant, partout (conception synchrone).

Motifs réutilisables

```
-- Détecteur de front (1 cycle)
process (clk) begin
    if rising_edge(clk) then prec <= sig; end if;
end process;
front <= sig and not prec;

-- Synchroniseur (entrée externe)
process (clk) begin
    if rising_edge(clk) then s0 <= ext; s1 <= s0; end if;
end process;

-- Diviseur / tick périodique
if cpt = F_CLK/2 - 1 then cpt <= (others=>'0'); tick <= '1';
else cpt <= cpt + 1; tick <= '0'; end if;

-- FSM
type t_etat is (REPOS, TRAVAIL);
signal etat : t_etat := REPOS;
process (clk) begin
    if rising_edge(clk) then
        case etat is
            when REPOS => if go then etat <= TRAVAIL; end if;
            when TRAVAIL => if fini then etat <= REPOS; end if;
        end case;
    end if;
end process;
sortie <= '1' when etat = TRAVAIL else '0'; -- Moore
```

Instanciation

```
u_bloc : entity work.mon_bloc
    generic map ( LARGEUR => 16 )
    port map ( clk => clk, reset => rst, d => data_in, q => data_out );
```

Testbench (obligatoire avant synthèse)

```
entity tb is end entity;                                -- pas de ports
architecture sim of tb is
    signal clk : std_logic := '0';
    signal fini : boolean := false;
begin
    dut : entity work.mon_bloc port map (...);
    clk <= not clk after 5 ns when not fini else '0';    -- 100 MHz

    stim : process
    begin
        wait for 20 ns;
        assert q = x"00" report "message d'erreur" severity error;
        report "TESTS OK"; fini <= true; wait;
    end process;
end architecture;
```

Sévérités

note **warning** **error** **failure** (stoppe la simu)

Synthétisable ou non

✓ Synthétisable

✗ Simulation seulement

if case for..generate

wait for 10 ns

rising_edge(clk)

after 5 ns dans une affectation

opérateurs logiques/arith.

report , assert (ignorés)

division par 2ⁿ (décalage)

division/modulo quelconque

GHDL en 4 commandes

```
ghdl -a --std=08 design.vhd tb.vhd    # analyser (sources AVANT testbench)
ghdl -e --std=08 tb                    # élaborer
ghdl -r --std=08 tb --wave=o.ghw      # simuler
gtkwave o.ghw                          # voir les chronogrammes
```

En ligne, sans rien installer : edaplayground.com (GHDL + EPWave).